# Lab 19: The refund failed. The customer was told it worked.

Topic 2: Tool Design & MCP Integration · Task 2.2: Implement structured error responses for MCP tools
Runs on Simulator 1: Customer Support Resolution Agent · Run in class
Source: SkillCertPro practice exam Part 1, Q29 (unofficial third-party question)

## Commit to an answer first

A refund operation sometimes fails because the order is outside the return window, but the agent treats the MCP response as a successful refund and tells the customer it was completed. What change should this team make to communicate the tool failure correctly?

**A.** Return a CallToolResult with isError: true and include structured failure details in the result content.

**B.** Return a normal tool result containing an error field and ask the agent to infer failure from the content.

**C.** Return a CallToolResult with isError: false and include structured failure details in the result content.

**D.** Raise a protocol-level server exception and include structured failure details in the exception message.

## Simulate the solution

```
# In the browser: open Simulator 1, then Lab 19
cd labs
uv run serve.py

# In Claude Code, from the labs folder
/lab 19

# In a terminal, from simulators/simulator-1-customer-support/
uv run support_agent.py --case CS-11
```

## What you should see

The refund on ORD-5508 is never issued, and the reply never claims it was.

The agent doesn't claim a refund. It may refuse from the policy, or call process_refund and read a business error. Either way, no refund is issued.

Open `simulators/simulator-1-customer-support/evidence/` and find it yourself.

## The answer

**A.** Return isError: true with structured failure details.

In MCP a tool call can succeed at the protocol level yet fail at the business-logic level. The pattern is to return a CallToolResult with isError: true and put useful recovery context in the content. Failures differ (policy violation, invalid date, missing order, permission denied), and structured details let the agent pick the next step: explain the rule, correct the input, retry a transient error, or escalate. Spelling and value both matter: camelCase isError , set to true .

## Why the others fail

**B.** Not the documented MCP error signal. The response still looks like a success.

**C.** isError: false explicitly signals success.

**D.** Protocol errors are for protocol failures. A business-rule failure belongs in the result, where the agent can reason about recovery.

## Check yourself

- the error carries errorCategory, isRetryable and a customer_message
- the reply explains the closed window instead of claiming success

## Now change one thing

Copy a file in cases/ to a new CS id, change the customer's message and expected disposition, rerun with --case, then score it with evaluate.py.

Third-party practice material from SkillCertPro, reconstructed from a video transcript in the Tertiary Markdown Vault. Not a real exam item. Wording may differ from the original.